

Theory of Computation

Lecture 3: Deterministic Finite Automata

Sheikh Azizul Hakim

Lecturer, Department of CSE, BUET

Last compiled: June 28, 2026

How do we describe finite memory precisely?

3A. The Concept of State & Mathematical Models

Where We Are in the Story

Lecture	Question	Main idea
1	Why study computation?	Limits of machines
2	What are problems?	Languages and finite memory intuition
3	How do we specify memory?	Deterministic finite automata
4	What if machines can guess?	Nondeterminism (Next)

Today we turn intuition into a precise mathematical object.

What is a “State”?

Before formalizing automata, we must understand a **state**.

A system interacts with a sequence of events over time. It cannot (and should not) remember every single event that ever happened.

Core Concept

A **state** is a compressed summary of the past. It captures *exactly* the information needed to determine future behavior, ignoring irrelevant history.

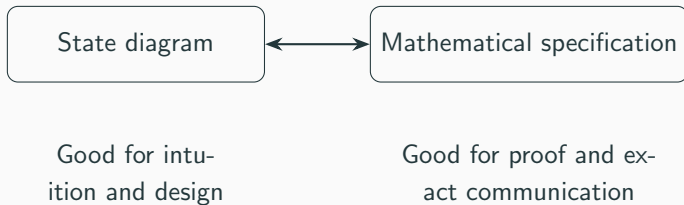
Suppose you designed a password validator or a tokenizer based on states.

How can you communicate the design to someone else?

- A drawing can be intuitive.
- A mathematical specification is precise.
- A table can define one part of the mathematics.

In theory, we need descriptions that are clear, compact, and unambiguous.

Two Views of the Same Machine



A transition table is simply one convenient way to define the transition function.

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,
3. a rule for moving between states,
4. a start state,
5. a set of accepting states.

So we write

$$M = (Q, \Sigma, \delta, q_0, F).$$

Definition: Deterministic Finite Automaton

A **deterministic finite automaton**, or DFA, is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F),$$

where

- Q is a finite set of states,
- Σ is a finite input alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of accepting states.

The Running Example: Even Number of 1's

Language we want to recognize:

$$\text{EVEN}_1 = \{w \in \{0, 1\}^* : w \text{ contains an even number of 1's}\}.$$

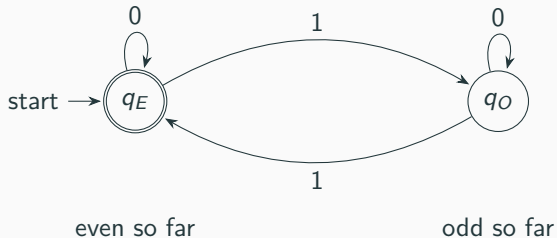
Members: $\varepsilon, 0, 11, 1010$ Non-members: $1, 10, 111$.

What information must the machine remember?

Only whether the number of 1's seen so far is even or odd.

q_E : even number of 1's seen, q_O : odd number of 1's seen.

The Same Machine as a Diagram



States are not merely circles; they encode the information remembered by the machine.

The Same Machine as Mathematics

$$Q = \{q_E, q_O\}, \quad \Sigma = \{0, 1\}, \quad q_0 = q_E, \quad F = \{q_E\}.$$

The transition function δ is defined by

δ	0	1
q_E	q_E	q_O
q_O	q_O	q_E

This table is not a third representation; it is simply a compact definition of δ .

Why Is It Called Deterministic?

For every pair

$$(q, a) \in Q \times \Sigma,$$

there is exactly one next state

$$\delta(q, a) \in Q.$$

No guessing.

No ambiguity.

No missing transition.

Exactly one next state.

A DFA is finite memory plus a deterministic update rule.

3B. Acceptance, Rejection, and Languages

Formal Acceptance

Let

$$M = (Q, \Sigma, \delta, q_0, F)$$

be a DFA, and let

$$w = w_1 w_2 \cdots w_n$$

be an input string over Σ .

The machine accepts w if there is a sequence of states

$$r_0, r_1, \dots, r_n$$

such that

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i \leq n-1$,
3. $r_n \in F$.

Language of a DFA

Let M be a DFA. The **language of** M is

$$L(M) = \{w \in \Sigma^* : M \text{ accepts } w\}.$$

A DFA M **accepts** or **recognizes** a language L when

$$L(M) = L.$$

So for the intended language L :

$$w \in L \Rightarrow M \text{ accepts } w, \quad w \notin L \Rightarrow M \text{ rejects } w.$$

What About the Empty String?

The empty string is

ε .

It has length zero.

If the input is ε , the DFA reads no symbols.

So it remains in the start state q_0 .

Therefore:

A DFA accepts ε exactly when $q_0 \in F$.

Three Similar-Looking Objects

These are different:

Notation	Meaning	Size
ε	the empty string	length 0
$\emptyset$	the empty language	0 strings
$\{\varepsilon\}$	language containing only the empty string	1 string

Common mistake:

$$\emptyset \neq \{\varepsilon\}.$$

Dead States

A dead state represents a situation from which success is impossible.

Once the machine enters a dead state, it stays there forever.



Dead states ensure the automaton mathematically remains deterministic and complete.

3C. Worked Examples: Designing States

How to Design a DFA

A good design strategy:

1. Decide what information must be remembered.
2. Make one state for each possible memory situation.
3. Define how each input symbol updates that information.
4. Mark the accepting states.

The hardest step is almost always the first one.

What finite information is enough?

Example 1: Strings Containing 001

Language:

$$L = \{w \in \{0, 1\}^* : w \text{ contains } 001 \text{ as a substring}\}.$$

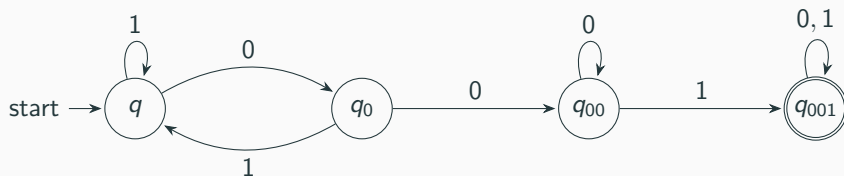
Members: 001, 1001, 0010 Non-members: ε , 00, 0101.

What should the states remember?

The longest suffix of the input so far that is also a prefix of 001.

- nothing useful yet,
- just saw 0,
- just saw 00,
- have seen 001.

DFA for Strings Containing 001



A doubly circled state means an accepting state.

Example 2: Binary Numbers Divisible by 3

Language:

$$L = \{w \in \{0,1\}^+ : w \text{ is the binary representation of a number divisible by 3}\}.$$

Members: 0, 11, 110, 1001 Non-members: ε , 1, 10, 100.

At first this looks arithmetic, not automata. But we only need to remember the remainder modulo 3.

q_s : no bit read yet, q_0 : rem 0, q_1 : rem 1, q_2 : rem 2.

Special Note: * vs +

Σ^* means zero or more symbols, so $\varepsilon \in \Sigma^*$. Σ^+ means one or more symbols, so $\varepsilon \notin \Sigma^+$.

Updating Remainders

If the current value has remainder r modulo 3, and we append bit b , the new value is

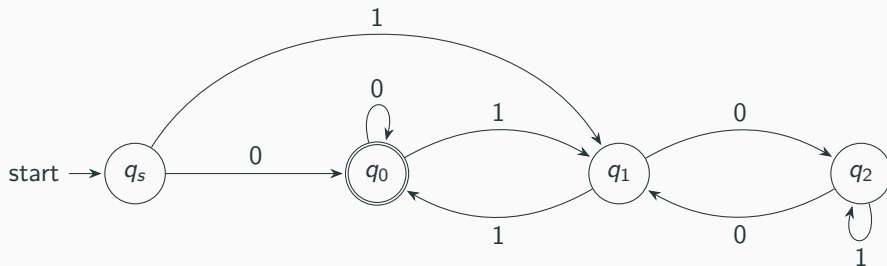
$$2r + b \pmod{3}.$$

Why? Appending a bit to the right doubles the old binary number and adds the new bit.

So the transition rule is

$$r \mapsto 2r + b \pmod{3}.$$

DFA for Divisibility by 3



Accept when at least one bit has been read and the remainder is 0.

Example 3: Valid Football Scores

Language over $\Sigma = \{0, d, -\}$, where d means any nonzero digit:

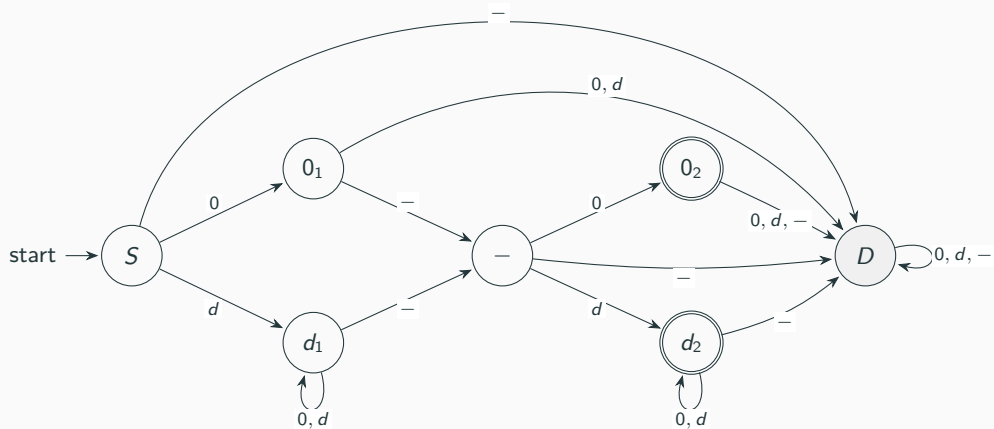
$$L = \{x - y : x, y \text{ are nonnegative integers with no leading zeros}\}.$$

Members: 0-0, d-0, d0-dd Non-members: ε , 00-d, d-, d--0.

What must the state remember?

- Are we reading the first score or the second score?
- Did the current score start with 0 or with a nonzero digit?
- Have we already used the single hyphen?

DFA for Valid Football Scores



3D. Advanced Examples: Pushing the Limits

Advanced Example 1: The Product Construction Concept

Language over $\Sigma = \{0, 1\}$:

$L = \{w : w \text{ has an even number of 0's } \mathbf{AND} \text{ its length is a multiple of 3}\}.$

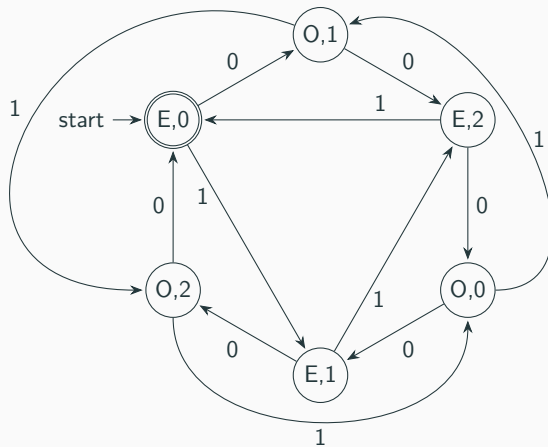
Members: $\varepsilon, 111, 001$ Non-members: $0, 11, 011$.

We must track **two** independent pieces of information simultaneously:

1. Parity of 0's (Even or Odd) $\rightarrow$ 2 possibilities
2. Length modulo 3 (0, 1, or 2) $\rightarrow$ 3 possibilities

Total states required: $2 \times 3 = 6$. A state is a pair (p, l) .

DFA for Multiple Conditions



Advanced Example 2: The k -th Symbol from the End

Language:

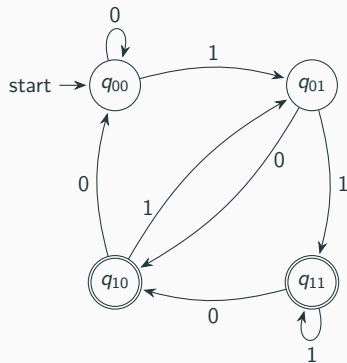
$$L = \{w \in \{0,1\}^* : \text{the 2nd symbol from the right is a 1}\}.$$

Members: 10, 011, 110, 0011 Non-members: ϵ , 0, 1, 100.

What must the state remember? It must act like a sliding window, remembering the **last 2 symbols** seen. States: $q_{00}, q_{01}, q_{10}, q_{11}$.

If we are in q_{01} (last two symbols were 0, 1) and we read a 0, our new “last two symbols” are 1, 0. Thus, $q_{01} \xrightarrow{0} q_{10}$.

DFA for the 2nd Symbol from the Right



Initial state q_{00} pretends we have seen leading zeros.

Advanced Example 3: Pattern Interference

Language over $\Sigma = \{a, b\}$:

$L = \{w : w \text{ contains } ab \text{ as a substring, but does **not** contain } ba\}$.

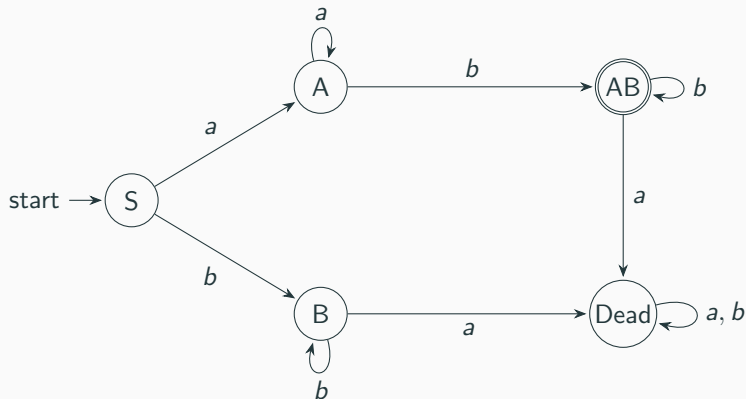
Members: $ab, aab, abb, aaabbb$ Non-members: $\varepsilon, a, b, aba, ba$.

Think about the structure of valid strings:

- They must eventually contain ab .
- Once a b occurs, no a can ever follow it (otherwise we get ba).
- Therefore, valid strings look exactly like a^+b^+ .

States needed: start, reading a 's, started with b 's, seen ab , and dead after ba .

DFA for Pattern Interference



Notice how starting with b means the string can never contain ab without first producing ba .

Advanced Example 4: No Symbol Repeated

Language over $\Sigma = \{a, b, c\}$:

$$L = \{w \in \{a, b, c\}^* : \text{no symbol appears more than once in } w\}.$$

Members: $\varepsilon, a, ba, cab, bca$ Non-members: $aa, aba, cabc, bb$.

What must the state remember?

The set of symbols seen so far.

$\emptyset, a, b, c, ab, bc, ca, abc$

plus one dead state D for a repeated symbol.

DFA for No Symbol Repeated

State x means the symbols in x are already encountered. All states except D are accepting.

State	on a	on b	on c
$\emptyset$	a	b	c
a	D	ab	ac
b	ab	D	bc
c	ac	bc	D
ab	D	D	abc
ac	D	abc	D
bc	abc	D	D
abc	D	D	D
D	D	D	D

Advanced Example 5: Lexical Analysis (C-Style Comments)

Language over $\Sigma = \{/, *, x\}$ (where x is any other char):

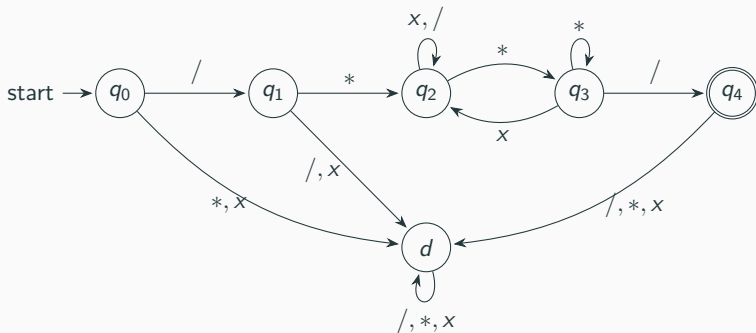
$L = \{w : w \text{ is exactly one valid block comment starting with } /* \text{ and ending with } */\}.$

Members: `/**/`, `/*x*/`, `/*/x**/` Non-members: `/*`, `/x*/`, `/*x*/x`, `/*x**/*/`.

Valid block comment

A valid block comment starts with `/*`, ends at the first later `*/`, and has no extra symbols after that closing delimiter.

DFA for C-Style Comments



This finite memory tracking is exactly how compilers strip comments from source code!

What You Should Be Able to Do Now

Given a DFA, you should be able to:

- identify $Q, \Sigma, \delta, q_0, F$,
- explain what each state remembers,
- trace an input string and decide if it is accepted,
- describe the language recognized by the DFA.

Given a language description, you should begin by asking:

What finite information is enough?

A DFA is not merely a diagram.

**It is a mathematical object that computes using finite
memory.**